

VPA A-Level CS — Weeks 10–18: Hardware, System Software & Security

AS Level Semester 1 — Weeks 10 – 18

Hardware, System Software & Security 硬件、系统软件与安全 (Yìngjiàn, xìtǒng ruǎnjiàn yǔ ānquán)

Syllabus Coverage: 1.4 Processor Fundamentals, 1.5 System Software, 1.6 Security, 1.7 Ethics

Paper: Paper 1 (Theory Fundamentals)

Duration: 9 weeks (Weeks 10 – 18)

Assessment: System Software Essay (W12), Security Case Study (W15), End-of-Semester Mock (W18)

Week-by-Week Scheme of Work

Week 10: Processor Fundamentals — Machine Architecture 处理器基础：机器架构

Lesson 1 (40 min): Machine Architecture & Instruction Sets 机器架构与指令集

Learning Objectives:

- ☐ Describe basic machine architecture 描述基本机器架构
- ☐ Explain register functions 解释寄存器功能
- ☐ Understand instruction set categories 理解指令集类别

Starter (5 min): CPU speaks "machine code" — only 0s and 1s. How does it know what each binary pattern means?

Main Activity (20 min):

- ☐ Machine code: Binary instructions executed directly by CPU. Architecture-specific.
机器码：CPU直接执行的二进制指令。
- ☐ Instruction set: Complete set of commands a CPU understands. Examples: x86, ARM, RISC-V, MIPS.
- ☐ CISC (Complex Instruction Set Computer): Many complex instructions. x86. Variable length. Powerful but harder to optimise.
- ☐ RISC (Reduced Instruction Set Computer): Few simple instructions. ARM, RISC-V. Fixed length. Faster per instruction, better pipelining. 精简指令集：指令少而简单，执行更快。
- ☐ Registers: Fast storage inside CPU. CIE requires:
 - PC (Program Counter): Address of next instruction
 - CIR (Current Instruction Register): Current instruction being executed
 - MAR (Memory Address Register): Address being accessed
 - MDR (Memory Data Register): Data being transferred
 - Acc (Accumulator): Stores results of ALU operations
 - IX (Index Register): Used for indexed addressing
 - SR (Status Register): Flags (Zero, Negative, Overflow, Carry)

☐ Instruction categories: Data movement (LOAD, STORE), Arithmetic (ADD, SUB), Logical (AND, OR, NOT), Control (JUMP, HALT, COMPARE)

Activity (10 min): Given register table, identify which register changes during each FDE stage.

Plenary (5 min): "Why does Apple use ARM instead of x86 for Macs now?" Discuss: power efficiency, RISC advantages, custom silicon.

Homework: Research: Find instruction set of a smartphone CPU. Is it RISC or CISC?

Bilingual Keywords:

- ☐ Instruction set 指令集 (zhǐlǐng jí)
- ☐ Register 寄存器 (jìcúnxì)
- ☐ Accumulator 累加器 (lěijiā qì)
- ☐ Status register 状态寄存器 (zhuàngtài jìcúnxì)

Lesson 2 (40 min): Addressing Modes 寻址方式

Learning Objectives:

- ☐ Explain immediate, direct, indirect, and indexed addressing 解释立即、直接、间接和变址寻址
- ☐ Identify addressing mode from assembly code 从汇编代码中识别寻址方式
- ☐ Compare efficiency of different modes 比较不同寻址方式的效率

Starter (5 min): "ADD 5" vs "ADD [5]" vs "ADD [R1]" — what's the difference in what gets added?

Main Activity (20 min):

- ☐ Immediate addressing: Operand is the value itself. ADD #5 → Add 5 to Acc. Fast, no memory access.
立即寻址: 操作数就是数值本身。
- ☐ Direct addressing: Operand is memory address. ADD 100 → Add contents of memory location 100. One memory access.
直接寻址: 操作数是内存地址。
- ☐ Indirect addressing: Operand is address of address. ADD [100] → Memory 100 contains address; fetch data from that address. Two memory accesses. 间接寻址: 操作数是地址的地址。
- ☐ Indexed addressing: Address = base + index register. LOAD 200, IX → Load from memory 200 + IX. Used for arrays.
变址寻址: 地址=基址+变址寄存器。
- ☐ Comparison table: Speed, flexibility, use cases.
- ☐ Worked examples: For each instruction, identify addressing mode and number of memory accesses.

Activity (10 min): Worksheet: 8 instructions, identify addressing mode, trace data flow.

Plenary (5 min): "Why is indexed addressing essential for array processing?" Discuss: loops, sequential access, pointer arithmetic.

Homework: Write 4 instructions using different addressing modes. Explain what each does.

Common Misconceptions:

- ☐ ☐ "Direct and immediate are the same" — Immediate: value IS the data. Direct: value IS the address.
- ☐ ☐ "Indirect addressing is too slow to be useful" — Used for pointers, linked lists, dynamic memory.

Lesson 3 (40 min): Assembly Language Introduction 汇编语言入门

Learning Objectives:

- ☐ Convert between machine code and assembly 在机器码和汇编之间转换
- ☐ Write simple assembly programs 编写简单汇编程序
- ☐ Trace assembly program execution 追踪汇编程序执行

Starter (5 min): Show binary: 11010011. What does it mean? Without knowing instruction set, it's meaningless. Assembly gives meaning.

Main Activity (20 min):

☐ Assembly: Human-readable mnemonics for machine code. One-to-one correspondence (usually).
汇编语言: 机器码的人类可读助记符。

☐ CIE pseudocode assembly style:

LDM #5 — Load immediate 5 into Acc

LDD 100 — Load direct from address 100

ADD #3 — Add immediate 3

SUB 200 — Subtract contents of address 200

STO 300 — Store Acc to address 300

JMP 50 — Jump to address 50

CMP #0 — Compare Acc with 0 (set flags)

JPE 80 — Jump if equal to 80

AND #15 — Logical AND

OR #8 — Logical OR

XOR #255 — Exclusive OR

LSL #2 — Logical shift left by 2

LSR #1 — Logical shift right by 1

IN / OUT — Input/Output

END — Halt

☐ Simple program: Input number, double it, output result.

IN → STO 100 → LDM #2 → MUL 100 → OUT → END

☐ Trace table: Track PC, Acc, memory contents through execution.

Practical (10 min): Use online simulator (e.g., CIE Assembly Simulator or Little Man Computer equivalent). Students run simple program.

Plenary (5 min): "Why do programmers rarely write assembly today?" Discuss: compilers, productivity, portability.

Homework: Write assembly program to calculate $3 \times (\text{input} + 5)$. Produce trace table.

Differentiation:

Bronze: Complete trace table for given program

Silver: Write program from pseudocode description

Gold: Write program with conditional logic (if input > 10, output "BIG"; else "SMALL")

Week 11: Assembly Language — Programs & Trace Tables 汇编语言: 程序与追踪表

Lesson 1 (40 min): Control Structures in Assembly 汇编中的控制结构

Learning Objectives:

☐ Implement selection in assembly 在汇编中实现选择结构

☐ Implement iteration in assembly 在汇编中实现循环结构

☐ Use flags and conditional jumps 使用标志和条件跳转

Starter (5 min): In Python: "if x > 0: print('positive')". How would CPU do this with only jumps and comparisons?

Main Activity (20 min):

☐ Selection (IF): Compare → Set flags → Conditional jump

```

IN // Input to Acc
CMP #0 // Compare with 0
JPE ZERO // Jump if equal to ZERO
JPG POS // Jump if positive to POS
JPL NEG // Jump if negative to NEG
ZERO: LDM #0 → JMP END
POS: LDM #1 → JMP END
NEG: LDM #-1
END: OUT → END

```

☐ Iteration (WHILE/FOR): Jump back to start if condition met

```

LDM #1 // Counter = 1
LOOP: CMP #10
JPG END // If counter > 10, exit
OUT // Output counter
ADD #1 // Increment
JMP LOOP // Repeat
END: END

```

☐ Flags: Zero (Z), Negative (N), Overflow (V), Carry (C). Set by CMP, ADD, SUB, etc.

☐ Exam tip: Trace tables MUST show Acc, IX, PC, and memory for each instruction.

Activity (10 min): Complete trace table for counting program. Identify where loop occurs.

Plenary (5 min): "What's the difference between JPE, JPG, JPL, JPN?" Mnemonic: E=Equal, G=Greater, L=Less, N=Not equal.

Homework: Write assembly program to sum numbers 1 to N (input N). Provide trace table for N=3.

Bilingual Keywords:

- ☐ Jump 跳转 (tiàozhuǎn)
- ☐ Conditional 条件的 (tiáojiàn de)
- ☐ Flag 标志 (biāozhì)
- ☐ Loop 循环 (xúnhuán)

Lesson 2 (40 min): Subroutines & the Stack 子程序与栈

Learning Objectives:

- ☐ Explain subroutine purpose and implementation 解释子程序的目的和实现
- ☐ Describe stack operations (push, pop) 描述栈操作 (压入、弹出)
- ☐ Trace subroutine execution with stack 用栈追踪子程序执行

Starter (5 min): In Python, you use functions. In assembly, how do you "call" a block of code and return?

Main Activity (20 min):

- ☐ Subroutine: Reusable block of code. CALL to enter, RETURN to exit. 子程序: 可重用的代码块。
- ☐ Problem: When subroutine calls another subroutine, how does CPU know where to return? → Stack.
- ☐ Stack: LIFO (Last In, First Out) data structure. Push adds to top, pop removes from top. 栈: 后进先出数据结构。
- ☐ CALL instruction: Push return address to stack, jump to subroutine.
- ☐ RETURN instruction: Pop address from stack, jump back.

☐ Stack overflow: Stack exceeds allocated memory. Caused by infinite recursion or deep calls.
栈溢出: 栈超出分配内存。

☐ Worked example: Main calls Sub1, Sub1 calls Sub2. Trace stack contents.

Stack: [Main return] → push [Sub1 return] → push [Sub2 return]

Return: pop → Sub2 returns to Sub1 → pop → Sub1 returns to Main

Activity (10 min): Given nested subroutine calls, draw stack at each point.

Plenary (5 min): "What happens if RETURN is called without matching CALL?" Discuss: stack underflow, program crash.

Homework: Explain why recursion needs a stack. Draw stack for factorial(3) with recursive calls.

Common Misconceptions:

☐ ☐ "Stack and queue are the same" — Stack = LIFO, Queue = FIFO

☐ ☐ "Stack is stored in RAM" — Yes, but managed specially. Stack pointer register tracks top.

Lesson 3 (40 min): Assembly Practice & Exam Technique 汇编练习与考试技巧

Learning Objectives:

☐ Complete assembly programming tasks under timed conditions 在限时条件下完成汇编编程任务

☐ Identify common trace table errors 识别常见追踪表错误

☐ Apply mark schemes to maximise marks 应用评分标准以获取最高分数

Starter (5 min): Quick fire: "What does LDD 50 do?" "What does JPE mean?" "What flag does CMP set?"

Main Activity (20 min):

☐ Exam-style question (6 marks): Write assembly program to find maximum of 3 numbers.

☐ Students attempt individually (10 min).

☐ Peer review using mark scheme:

— Correct input (1 mark)

— Comparison logic (2 marks)

— Correct output (1 mark)

— Program halts (1 mark)

— Appropriate use of labels/comments (1 mark)

☐ Common errors from teacher experience:

— Forgetting to store intermediate results

— Wrong conditional jump (JPE instead of JPG)

— Not initialising accumulator

— Infinite loops (missing exit condition)

— Trace tables missing PC values

Activity (10 min): Given incorrect trace table, identify and correct 5 errors.

Plenary (5 min): Assembly programming checklist: Before submitting, check... (students generate list)

Homework: Past paper question: 9618/11 ON22 Q3. Complete in timed conditions (15 min). Self-mark.

EAL Support:

☐ Provide bilingual instruction glossary for assembly terms

☐ Allow extra reading time for exam questions

☐ Visual flowchart of program before writing assembly

Week 12: System Software — Operating Systems 系统软件：操作系统

Lesson 1 (40 min): OS Functions & Types 操作系统功能与类型

Learning Objectives:

- ☐ Define operating system and list its functions 定义操作系统并列出其功能
- ☐ Compare batch, interactive, real-time, and distributed OS 比较批处理、交互式、实时和分布式操作系统
- ☐ Explain role of OS in resource management 解释操作系统在资源管理中的作用

Starter (5 min): What happens when you turn on your computer before you see the desktop? What runs first?

Main Activity (20 min):

☐ Operating System: System software managing hardware resources and providing services.
操作系统：管理系统硬件资源并提供服务的系统软件。

☐ Core functions:

1. Memory management: Allocates RAM to processes, handles virtual memory, paging, segmentation.
内存管理：分配RAM，处理虚拟内存。
2. Processor scheduling: Decides which process runs when. Prevents one program monopolising CPU.
处理器调度：决定哪个进程何时运行。
3. Peripheral management: Controls I/O devices via drivers. 外设管理：通过驱动程序控制输入输出设备。
4. File management: Organises storage, maintains directory structure, handles permissions.
文件管理：组织存储，维护目录结构。
5. Security: User authentication, access control, encryption. 安全：用户认证，访问控制。
6. User interface: GUI, CLI, voice, touch. 用户界面：图形界面、命令行界面。

☐ OS types:

- Batch: Jobs collected, processed without user interaction. Payroll systems. 批处理：作业批量处理，无需用户交互。
- Interactive: User interacts directly. Desktop OS, mobile OS. 交互式：用户直接交互。
- Real-time: Guaranteed response time. Industrial control, medical devices, avionics. Hard vs soft real-time.
实时：保证响应时间。
- Distributed: Multiple computers work as single system. Cloud computing, blockchain. 分布式：多台计算机协同工作。

Activity (10 min): Match OS type to scenario: (a) air traffic control, (b) monthly payroll, (c) iPhone, (d) Google search.

Plenary (5 min): "Why can't a batch OS be used for a self-driving car?" Discuss: need immediate response, safety-critical.

Homework: Research: What OS runs on your phone? Your smartwatch? Your school's servers? Classify each.

Bilingual Keywords:

- ☐ Operating system 操作系统 (cāozuò xìtǒng)
- ☐ Kernel 内核 (nèihé)
- ☐ Process 进程 (jìnchéng)
- ☐ Driver 驱动程序 (qūdòng chéngxù)

Lesson 2 (40 min): Memory Management 内存管理

Learning Objectives:

- ☐ Explain paging and segmentation 解释分页和分段
- ☐ Describe virtual memory 描述虚拟内存
- ☐ Discuss memory allocation strategies 讨论内存分配策略

Starter (5 min): Your phone has 8GB RAM but runs 20 apps. How? Are they all in RAM at once?

Main Activity (20 min):

☐ Paging: Memory divided into fixed-size pages. Programs allocated page frames. OS maintains page table.
分页: 内存划分为固定大小的页。

— Advantages: No external fragmentation, easy to manage

— Disadvantages: Internal fragmentation (last page may not be full)

☐ Segmentation: Memory divided by logical units (code, data, stack). Variable sizes. 分段: 按逻辑单元划分内存。

— Advantages: Matches program structure, shared segments

— Disadvantages: External fragmentation (gaps between segments)

☐ Virtual memory: Uses disk space as extension of RAM. Pages swapped in/out. 虚拟内存: 使用磁盘空间扩展RAM。

— Page fault: Required page not in RAM. OS fetches from disk (slow!).

— Thrashing: Excessive page swapping. System becomes unusable.

☐ Comparison: Paging vs Segmentation table (size, fragmentation, complexity, protection).

Activity (10 min): Given memory layout diagram, identify pages/segments, calculate fragmentation.

Plenary (5 min): "Why do gaming PCs need lots of RAM to avoid lag?" Discuss: reduces page faults, virtual memory too slow.

Homework: Explain paging to a Year 7 student using analogy. 用类比向初中生解释分页。

Common Misconceptions:

☐ ☐ "Virtual memory is as fast as RAM" — Disk is 100,000× slower. Virtual memory prevents crashes but causes slowdown.

☐ ☐ "Paging and segmentation are the same" — Paging: fixed size, invisible to programmer. Segmentation: variable size, logical units.

Lesson 3 (40 min): Scheduling Algorithms 调度算法

Learning Objectives:

☐ Explain purpose of CPU scheduling 解释CPU调度的目的

☐ Describe FCFS, SJF, Round Robin, Priority scheduling 描述先来先服务、短作业优先、时间片轮转、优先级调度

☐ Compare scheduling algorithms 比较调度算法

Starter (5 min): 5 students at canteen. One orders complex meal, others simple. How should server prioritise?

Main Activity (20 min):

☐ Scheduling: OS decides which ready process gets CPU. Affects response time, throughput, fairness.
调度: 操作系统决定哪个就绪进程获得CPU。

☐ FCFS (First Come First Served): Simple queue. Fair but convoy effect (long job delays short jobs).
先来先服务: 简单队列。

☐ SJF (Shortest Job First): Minimises average waiting time. Requires knowing job length. Risk of starvation for long jobs. 短作业优先: 最小化平均等待时间。

☐ Round Robin: Fixed time quantum. Each process gets equal share. Good for time-sharing. Quantum too small = overhead; too large = FCFS. 时间片轮转: 固定时间片, 轮流执行。

☐ Priority Scheduling: Higher priority processes first. Can preempt lower priority. Risk of starvation.
优先级调度: 高优先级进程先执行。

☐ Worked examples: Calculate waiting time and turnaround time for each algorithm with same job set.

Activity (10 min): Given 4 processes with burst times, calculate average waiting time for FCFS, SJF, Round Robin.

Plenary (5 min): "Which algorithm is best for a hospital emergency room system?" Discuss: priority with aging to prevent starvation.

Homework: Create decision tree: "Which scheduling algorithm for..." (a) supercomputer, (b) smartphone, (c) stock trading.

Week 12 Assessment: System Software Essay (20 minutes)

- ☐ "Compare paging and segmentation as memory management techniques. Discuss advantages and disadvantages of each." (9 marks)
- ☐ OR
- ☐ "Explain why a real-time operating system is necessary for an autonomous vehicle." (6 marks)
- ☐ Marked with CIE mark scheme. Focus: structure, technical accuracy, comparison, conclusion.

Week 13: Translators — Compilers, Interpreters, Assemblers

翻译器：编译器、解释器、汇编器

Lesson 1 (40 min): Translators Overview & Assemblers 翻译器概述与汇编器

Learning Objectives:

- ☐ Define translator and identify types 定义翻译器并识别类型
- ☐ Explain assembler function 解释汇编器的功能
- ☐ Compare one-pass and two-pass assemblers 比较单遍和双遍汇编器

Starter (5 min): You write Python code. CPU doesn't understand Python. What happens in between?

Main Activity (20 min):

- ☐ Translator: Converts source code to another form. 翻译器：将源代码转换为其他形式。
- ☐ Types:
 - Assembler: Assembly → Machine code. One-to-one (mostly). 汇编器：汇编语言→机器码。
 - Compiler: High-level language → Machine code (all at once). Produces executable. 编译器：高级语言→机器码（一次性）。
 - Interpreter: High-level language → Executed line by line. No separate executable. 解释器：高级语言→逐行执行。
- ☐ Assembler operation:
 - Symbol table: Maps labels to memory addresses.
 - Opcode table: Maps mnemonics to binary opcodes.
 - One-pass: Single scan. Forward references problematic.
 - Two-pass: First pass builds symbol table, second pass generates code. Handles forward references.
- ☐ Example: LOOP: ADD #1 → Assembler looks up ADD opcode, assigns address to LOOP, outputs machine code.

Activity (10 min): Manual assembly: Convert 3 assembly instructions to machine code using provided opcode table.

Plenary (5 min): "Why do assemblers need two passes?" Discuss: forward references, label resolution.

Homework: Research: What's the difference between assembler and linker? What does linker do?

Bilingual Keywords:

- ☐ Assembler 汇编器 (huìbiān qì)
- ☐ Compiler 编译器 (biānyì qì)
- ☐ Interpreter 解释器 (jiěshì qì)
- ☐ Source code 源代码 (yuándài mǎ)

Lesson 2 (40 min): Compilers — Lexical & Syntax Analysis 编译器：词法分析与语法分析

Learning Objectives:

- ☐ Explain compilation stages 解释编译阶段

☐ Describe lexical analysis 描述词法分析

☐ Describe syntax analysis 描述语法分析

Starter (5 min): Compiler error: "SyntaxError: invalid syntax". What did the compiler check? What stage failed?

Main Activity (20 min):

☐ Compilation stages: Lexical analysis → Syntax analysis → Semantic analysis → Code generation → Optimisation.
编译阶段: 词法分析→语法分析→语义分析→代码生成→优化。

☐ Lexical Analysis:

— Source code → Tokens. Removes whitespace/comments.

— Example: `x = 5 + 3` → [IDENTIFIER:x], [ASSIGN], [NUMBER:5], [PLUS], [NUMBER:3]

— Symbol table created for identifiers.

— Error example: `x$ = 5` → "\$" not valid token → lexical error.

☐ Syntax Analysis:

— Tokens checked against grammar rules. Builds parse tree/AST.

— Grammar rules define valid language structure.

— Example: `if x > 5 then` → Valid. `if x > 5` → Missing THEN → syntax error.

— Parse tree shows hierarchical structure of statement.

☐ Error types: Lexical (unrecognised symbols), Syntax (invalid structure), Semantic (valid syntax but meaningless, e.g., type mismatch).

Activity (10 min): Given code snippets, identify lexical vs syntax vs semantic errors.

Plenary (5 min): "Why must lexical analysis happen before syntax analysis?" Discuss: tokens are input to parser.

Homework: Draw parse tree for: `y = (a + b) * c`. Identify tokens first.

Differentiation:

Bronze: Identify token types in simple statements

Silver: Explain why code produces syntax error

Gold: Design simple grammar for calculator expressions (BNF notation)

Lesson 3 (40 min): Compilers vs Interpreters 编译器与解释器对比

Learning Objectives:

☐ Compare compilers and interpreters 比较编译器和解释器

☐ Give advantages and disadvantages of each 给出各自的优缺点

☐ Explain when each is appropriate 解释何时使用每种方式

Starter (5 min): Python (interpreted) vs C++ (compiled). When would you choose each? Speed? Ease?

Main Activity (20 min):

☐ Compiler:

— Translates entire program before execution

— Produces standalone executable

— Advantages: Faster execution, error detection before running, optimised code

— Disadvantages: Slower development cycle, platform-specific executable, harder debugging

— Examples: C, C++, Rust, Go, Fortran

☐ Interpreter:

— Translates and executes line by line

- No separate executable
- Advantages: Faster development, easier debugging, platform-independent
- Disadvantages: Slower execution, errors found at runtime, source code needed
- Examples: Python, JavaScript, Ruby, PHP

- ☐ Hybrid: Java compiles to bytecode, interpreted by JVM. Combines benefits. C# similar with CLR.
- ☐ Comparison table: Speed, debugging, portability, security, memory use.

Activity (10 min): Scenario cards: Choose translator for (a) spacecraft control, (b) web app, (c) OS kernel, (d) teaching beginners.

Plenary (5 min): "Why is Java called 'platform independent' if it needs JVM?" Discuss: bytecode runs on any JVM, not machine code.

Homework: Create Venn diagram: Compiler vs Interpreter. Include hybrid approaches in intersection.

Common Misconceptions:

- ☐ ☐ "Compiled languages are always faster" — Modern JIT compilation (JavaScript V8) narrows gap
- ☐ ☐ "Python is interpreted so it can't be used for serious applications" — Instagram, YouTube use Python extensively

Week 14: Security — Threats & Malware 安全：威胁与恶意软件

Lesson 1 (40 min): Types of Malware 恶意软件类型

Learning Objectives:

- ☐ Define malware and identify types 定义恶意软件并识别类型
- ☐ Explain how each malware type operates 解释每种恶意软件的运作方式
- ☐ Describe social engineering attacks 描述社会工程学攻击

Starter (5 min): News headline: "Major hospital hit by ransomware." What's ransomware? How does it work?

Main Activity (20 min):

- ☐ Malware = Malicious software. Designed to damage, disrupt, or gain unauthorised access. 恶意软件：旨在破坏、干扰或非法访问的软件。
- ☐ Virus: Attaches to legitimate program. Requires user action to spread. 病毒：依附于合法程序，需用户操作传播。
- ☐ Worm: Self-replicating, spreads through networks. No user action needed. 蠕虫：自我复制，通过网络传播。
- ☐ Trojan: Disguised as legitimate software. Creates backdoor. 木马：伪装成合法软件，创建后门。
- ☐ Ransomware: Encrypts files, demands payment for key. 勒索软件：加密文件，要求支付赎金。
- ☐ Spyware: Secretly monitors activity, steals data. 间谍软件：秘密监控活动，窃取数据。
- ☐ Adware: Unwanted advertisements, often bundled. 广告软件：不需要的广告。
- ☐ Rootkit: Hides presence of malware, deep system access. Rootkit：隐藏恶意软件，深层系统访问。
- ☐ Social engineering: Manipulating people into revealing information. 社会工程学：操纵人们泄露信息。
- Phishing: Fake emails/websites. Spear phishing targets specific person.
- Pretexting: Creating false scenario to gain trust.
- Baiting: Offering something enticing (free USB drive).
- Quid pro quo: Offering service in exchange for information.

Activity (10 min): Case studies: Identify malware type and social engineering technique in 3 scenarios.

Plenary (5 min): "Why is social engineering often more effective than technical hacking?" Discuss: humans are weakest link.

Homework: Create infographic: 5 malware types with icons, symptoms, prevention tips. Bilingual.

Bilingual Keywords:

- ☐ Malware 恶意软件 (èyì ruǎnjiàn)
- ☐ Ransomware 勒索软件 (lèsuǒ ruǎnjiàn)
- ☐ Phishing 网络钓鱼 (wǎngluò diàoyú)
- ☐ Firewall 防火墙 (fánghuǒqiáng)

Lesson 2 (40 min): Security Measures 安全措施

Learning Objectives:

- ☐ Explain technical security measures 解释技术安全措施
- ☐ Describe physical security measures 描述物理安全措施
- ☐ Evaluate effectiveness of security strategies 评估安全策略的有效性

Starter (5 min): Your school has valuable data (grades, personal info). How is it protected? Brainstorm in groups.

Main Activity (20 min):

☐ Technical measures:

- Anti-malware software: Real-time scanning, heuristic detection, signature database.
反恶意软件: 实时扫描, 启发式检测。
- Firewalls: Packet filtering, stateful inspection, application layer filtering. Hardware vs software.
防火墙: 包过滤, 状态检测。
- Intrusion Detection Systems (IDS): Monitors network for suspicious activity. 入侵检测系统: 监控网络可疑活动。
- Intrusion Prevention Systems (IPS): Active blocking. 入侵防御系统: 主动阻止。
- Access control: Authentication (who you are) + Authorisation (what you can do). 访问控制: 认证+授权。
- Encryption: SSL/TLS for data in transit, full-disk encryption for data at rest.

☐ Physical measures:

- Biometric access (fingerprint, retina). 生物识别访问。
- CCTV, security guards, locked server rooms.
- Secure disposal (shredding, degaussing).

☐ Human/Procedural measures:

- Password policies, MFA, regular training, incident response plans.
- Backup strategy: 3-2-1 rule (3 copies, 2 media, 1 offsite).

Activity (10 min): Security audit: Evaluate VPA's security. Identify 3 strengths, 3 weaknesses, 3 improvements.

Plenary (5 min): "Can perfect security exist?" Discuss: security vs usability trade-off, zero-day exploits, human factor.

Homework: Write security policy for a small business. Include technical, physical, and procedural measures.

Differentiation:

Bronze: List 5 security measures with brief descriptions

Silver: Categorise measures and explain one in detail

Gold: Evaluate layered security (defence in depth) approach with real example

Lesson 3 (40 min): Encryption Fundamentals 加密基础

Learning Objectives:

- ☐ Explain purpose of encryption 解释加密的目的
- ☐ Describe symmetric and asymmetric encryption 描述对称和非对称加密

☐ Explain Caesar cipher and Vernam cipher 解释凯撒密码和弗纳姆密码

Starter (5 min): Send secret message to friend in class. How do you prevent others reading it?

Main Activity (20 min):

☐ Encryption: Scrambling data so only authorised parties can read. Plain text → Cipher text → Plain text.
加密: 打乱数据, 只有授权方可以读取。

☐ Symmetric encryption: Same key for encryption and decryption. Fast, but key distribution problem.
对称加密: 加密和解密使用相同密钥。

☐ Asymmetric encryption: Public key (encrypt) + Private key (decrypt). Solves key distribution. Slower.
非对称加密: 公钥加密, 私钥解密。

☐ Caesar cipher: Shift alphabet by fixed amount. Key = shift value. Weak (only 25 possibilities).

— Example: Shift 3. A→D, B→E, HELLO→KH00R

— Frequency analysis breaks it easily.

☐ Vernam cipher (One-time pad): Plain text XOR key (random, same length). Unbreakable if key truly random, used once, kept secret. 弗纳姆密码: 明文与随机密钥异或。

— Example: Plain = 0110, Key = 1011, Cipher = 1101

— Problems: Key distribution, key length, true randomness.

Activity (10 min): Encode message using Caesar cipher (shift 5). Partner decodes. Discuss how easy it was to break.

Plenary (5 min): "Why is Vernam cipher theoretically unbreakable?" Discuss: no pattern, all possibilities equally likely.

Homework: Research: How does modern AES encryption work? (Brief overview — CIE doesn't require detail). Compare with Caesar cipher.

Bilingual Keywords:

☐ Encryption 加密 (jiāmì)

☐ Decryption 解密 (jiěmì)

☐ Cipher 密码 (mìǎ)

☐ Key 密钥 (mìyuè)

☐ Plain text 明文 (míngwén)

☐ Cipher text 密文 (mìwén)

Week 15: Security Protocols & Digital Signatures 安全协议与数字签名

Lesson 1 (40 min): SSL/TLS, Digital Certificates & Firewalls SSL/TLS、数字证书与防火墙

Learning Objectives:

☐ Explain SSL/TLS purpose and operation 解释SSL/TLS的目的和运作

☐ Describe digital certificates 描述数字证书

☐ Explain firewall types and operation 解释防火墙类型和运作

Starter (5 min): Green padlock in browser. What does it mean? What happens if it's red/missing?

Main Activity (20 min):

☐ SSL/TLS (Secure Sockets Layer / Transport Layer Security): Encrypts data between browser and server.
SSL/TLS: 加密浏览器与服务器之间的数据。

— HTTPS = HTTP + SSL/TLS

— Handshake: Browser and server agree encryption method, exchange keys, verify identity.

— Protects against eavesdropping, tampering, man-in-the-middle attacks.

☐ Digital certificates: Verify identity of website/server. Issued by Certificate Authority (CA).
数字证书: 验证网站身份, 由证书颁发机构签发。

— Contains: Public key, owner identity, CA signature, validity period.

— Chain of trust: Root CA → Intermediate CA → Server certificate.

— Self-signed certificates: Not trusted by browsers (no CA verification).

☐ Firewalls:

— Packet filtering: Checks source/destination IP, port, protocol. Fast but limited.

— Stateful inspection: Tracks connection state. More secure.

— Application layer: Inspects actual data content. Most secure but slowest.

— Next-gen: Integrated IDS/IPS, deep packet inspection, AI-based detection.

Activity (10 min): Analyse certificate details of 3 websites (vpa.edu.cn, google.com, government site). Compare issuers, validity, encryption strength.

Plenary (5 min): "Why do some sites use HTTP instead of HTTPS?" Discuss: cost, legacy systems, lack of awareness.

Homework: Explain to parent why HTTPS matters. Write their likely questions and your answers.

Bilingual Keywords:

☐ SSL/TLS 安全套接层/传输层安全 (ānquán tàojiē céng/chuánshū céng ānquán)

☐ Certificate 证书 (zhèngshū)

☐ Certificate Authority 证书颁发机构 (zhèngshū bānfā jīgòu)

☐ HTTPS 超文本传输安全协议 (chāo wénběn chuánshū ānquán xiéyì)

Lesson 2 (40 min): Digital Signatures & Authentication 数字签名与认证

Learning Objectives:

☐ Explain digital signature purpose and process 解释数字签名的目的和过程

☐ Describe password policies and MFA 描述密码策略和多因素认证

☐ Discuss biometric authentication 讨论生物识别认证

Starter (5 min): You sign a document to prove it's from you. How do you "sign" a digital document?

Main Activity (20 min):

☐ Digital signature: Proves authenticity, integrity, non-repudiation. 数字签名: 证明真实性、完整性和不可否认性。

☐ Process:

1. Hash document (SHA-256) → fixed-length digest

2. Encrypt digest with sender's PRIVATE key → signature

3. Send document + signature

4. Receiver decrypts signature with sender's PUBLIC key → gets digest

5. Receiver hashes received document → gets digest

6. If digests match → document authentic and unmodified

☐ Non-repudiation: Sender cannot deny sending. Only they have private key. 不可否认性: 发送者不能否认发送。

☐ Password policies:

— Minimum length (12+ characters)

— Complexity (upper, lower, number, symbol)

— No dictionary words

— Regular changes (debated — NIST now recommends long memorable phrases)

— No reuse across sites

☐ Multi-Factor Authentication (MFA): Something you know (password) + something you have (phone) + something you are (fingerprint). 多因素认证: 所知+所有+所是。

☐ Biometrics: Fingerprint, face recognition, iris scan, voice. Convenient but irrevocable if compromised. 生物识别: 方便但泄露后不可更改。

Activity (10 min): Role-play: Bank security consultant. Recommend authentication for (a) online banking, (b) ATM, (c) phone banking.

Plenary (5 min): "Why is biometric authentication problematic if breached?" Discuss: can't change fingerprint like password.

Homework: Evaluate your own digital security. Score password strength, check MFA usage, list improvements.

Common Misconceptions:

☐ ☐ "Digital signature encrypts the whole document" — It encrypts the HASH, not the document

☐ ☐ "Biometrics are more secure than passwords" — Convenient but permanent; breach is catastrophic

Lesson 3 (40 min): Security Case Study & Review 安全案例研究与复习

Learning Objectives:

☐ Analyse real-world security incidents 分析真实安全事件

☐ Apply security knowledge to scenarios 将安全知识应用于场景

☐ Review Weeks 14 - 15 content 复习第14-15周内容

Starter (5 min): 2017 WannaCry ransomware. 200,000 computers in 150 countries. NHS hospitals affected. How? Why?

Main Activity (20 min):

☐ Case study: WannaCry

— Exploit: EternalBlue (NSA tool leaked by Shadow Brokers)

— Spread: Worm, no user interaction needed

— Impact: Encrypted files, \$300 Bitcoin ransom, hospitals cancelled operations

— Prevention: Patch management, network segmentation, backups, updated OS

— Lesson: Unpatched systems = vulnerability. Backups essential.

☐ Group analysis: Each group analyses different breach (Equifax 2017, Target 2013, SolarWinds 2020).

— What type of attack?

— What vulnerabilities existed?

— What prevention measures failed?

— What lessons learned?

Review (10 min): Security concept map: Link threats → vulnerabilities → impacts → countermeasures.

Plenary (5 min): "Is cybersecurity a technical problem or a human problem?" Structured debate (2 min each side).

Week 15 Assessment: Security Case Study Test (30 minutes)

☐ Scenario: "A school discovers ransomware on its file server. Student records are encrypted."

☐ Q1: Identify 3 types of malware that could cause this. (3 marks)

☐ Q2: Explain how the ransomware likely entered the system. (4 marks)

☐ Q3: Recommend technical, physical, and procedural measures to prevent future attacks. (9 marks)

☐ Q4: Explain why regular backups are essential. (4 marks)

☐ Q5: Describe how SSL/TLS would protect the school's online portal. (5 marks)

☐ Total: 25 marks — 10% of semester grade

Week 16: Ethics, Ownership & AI Ethics 伦理、所有权与人工智能伦理

Lesson 1 (40 min): Software Licensing & Intellectual Property 软件许可与知识产权

Learning Objectives:

- ☐ Compare software license types 比较软件许可类型
- ☐ Explain copyright, patents, trademarks 解释版权、专利和商标
- ☐ Discuss implications of software piracy 讨论软件盗版的 implications

Starter (5 min): You download cracked Photoshop. Is this wrong? Why? Who is harmed?

Main Activity (20 min):

- ☐ Proprietary/Commercial: Closed source, paid license. Examples: Microsoft Office, Adobe.
专有/商业软件: 闭源, 付费许可。
- ☐ Open Source: Source code available, free to use/modify/distribute. Examples: Linux, Firefox, Python.
开源软件: 源代码可用, 免费使用修改。
- GPL: Must share modifications. GPL: 必须分享修改。
- MIT: More permissive. MIT: 更宽松。
- Apache: Patent protection included. Apache: 包含专利保护。
- ☐ Freeware: Free to use, closed source. Examples: Zoom (basic), Skype.
- ☐ Shareware: Try before buy. Examples: WinRAR.
- ☐ Public Domain: No restrictions.
- ☐ Intellectual Property:
 - Copyright: Protects original works (software, music, writing). Automatic. Life + 70 years.
 - Patent: Protects inventions. Must apply. 20 years. Software patents controversial.
 - Trademark: Protects brand names/logos.
- ☐ Software piracy: Illegal copying/distribution. Consequences: fines, imprisonment, malware risk, harm to industry.

Activity (10 min): Licence matching: Given 6 software descriptions, identify licence type and justify.

Plenary (5 min): "Should all software be free and open source?" Structured debate.

Homework: Investigate: What licence is your favourite app/game under? What can/cannot you do with it?

Bilingual Keywords:

- ☐ License 许可证 (xǔkězhèng)
- ☐ Copyright 版权 (bǎnquán)
- ☐ Open source 开源 (kāiyuán)
- ☐ Proprietary 专有的 (zhuānyǒu de)

Lesson 2 (40 min): AI Ethics & Environmental Issues 人工智能伦理与环境问题

Learning Objectives:

- ☐ Discuss ethical implications of AI 讨论人工智能的伦理影响
- ☐ Explain bias in AI systems 解释AI系统中的偏见
- ☐ Discuss environmental impact of computing 讨论计算对环境的影响

Starter (5 min): AI hires job candidates. It rejects more women. Is this the AI's fault? Who is responsible?

Main Activity (20 min):

- ☐ AI Ethics issues:

— Bias: Training data reflects human biases. Amazon AI recruiting tool penalised women's resumes (trained on male-dominated data). 偏见: 训练数据反映人类偏见。

— Privacy: Facial recognition, data harvesting, surveillance capitalism. 隐私: 人脸识别, 数据收集。

— Accountability: Who is responsible when AI causes harm? Developer? Company? User? 责任: AI造成伤害时谁负责?

— Transparency: "Black box" AI decisions hard to explain. Need explainable AI (XAI). 透明度: 黑箱AI难以解释。

— Job displacement: Automation replaces workers. Need reskilling, UBI debates. 就业替代: 自动化取代工人。

— Autonomous weapons: Should AI decide to kill? International debates. 自主武器: AI应该决定是否杀戮吗?

☐ Environmental impact:

— Data centres consume 1% global electricity. 数据中心消耗全球1%电力。

— Bitcoin mining: Energy equivalent of small countries.

— E-waste: Toxic materials, recycling challenges.

— Carbon footprint of AI training: GPT-3 training = 5 cars' lifetime emissions.

— Solutions: Renewable energy, efficient algorithms, longer device lifespans, right to repair.

Activity (10 min): Structured debate: "AI development should be paused until ethical frameworks are established."

Plenary (5 min): "What can YOU do as a CS student to promote ethical technology?" Individual pledges.

Homework: Research one AI ethics case study. Prepare 2-minute presentation.

Differentiation:

Bronze: Identify 3 AI ethics issues from provided list

Silver: Explain one issue with real example

Gold: Propose policy solution for chosen issue; evaluate feasibility

Lesson 3 (40 min): Data Protection Act & Global Regulations 数据保护法与全球法规

Learning Objectives:

☐ Explain principles of data protection legislation 解释数据保护法规原则

☐ Compare GDPR, PIPL (China), and other frameworks 比较GDPR、中国《个人信息保护法》等框架

☐ Discuss implications for organisations 讨论对组织的影响

Starter (5 min): Your school has your name, photo, grades, health info. Who can see it? What if it's leaked?

Main Activity (20 min):

☐ Data Protection principles (common across frameworks):

1. Lawful, fair, transparent processing 合法、公正、透明处理

2. Purpose limitation (collect for specified purpose only) 目的限制

3. Data minimisation (collect only what's necessary) 数据最小化

4. Accuracy (keep up to date) 准确性

5. Storage limitation (don't keep forever) 存储限制

6. Integrity and confidentiality (security) 完整性和保密性

7. Accountability (organisation responsible) 问责制

☐ GDPR (EU): Strictest. Fines up to 4% global revenue. Right to be forgotten. Data portability. GDPR: 最严格, 最高罚款全球收入4%。

☐ PIPL (China): Personal Information Protection Law. Consent required. Cross-border transfer restrictions. 《个人信息保护法》: 需同意, 跨境传输限制。

☐ Implications for organisations:

— Data Protection Officer required for large organisations

- Privacy Impact Assessments
- Breach notification (72 hours under GDPR)
- Consent management systems

Activity (10 min): Case study: VPA wants to use facial recognition for attendance. Apply data protection principles. Is this lawful? What safeguards needed?

Plenary (5 min): "Does strong data protection harm innovation?" Discuss: compliance costs vs trust, long-term benefits.

Homework: Write data protection policy for a student app. Include: what data collected, why, how stored, how long kept, user rights.

Bilingual Keywords:

- ☐ GDPR 通用数据保护条例 (tōngyòng shùjù bǎohù tiáolì)
- ☐ PIPL 个人信息保护法 (gèrén xìnxī bǎohù fǎ)
- ☐ Consent 同意 (tóngyì)
- ☐ Data subject 数据主体 (shùjù zhǔtǐ)

Week 17: Programming Introduction — Pseudocode & Flowcharts 编程入门：伪代码与流程图

Lesson 1 (40 min): Algorithm Design & Pseudocode 算法设计与伪代码

Learning Objectives:

- ☐ Write algorithms using pseudocode 使用伪代码编写算法
- ☐ Apply stepwise refinement 应用逐步求精
- ☐ Use IPO charts 使用输入-处理-输出表

Starter (5 min): How do you make instant noodles? Write step-by-step instructions. Someone else follows them literally. What goes wrong?

Main Activity (20 min):

- ☐ Algorithm: Step-by-step solution to problem. Must be unambiguous, finite, effective. 算法：解决问题的逐步方案。
- ☐ Pseudocode: Language-neutral description of algorithm. CIE has specific conventions.
- ☐ CIE Pseudocode essentials:

DECLARE Counter : INTEGER

CONSTANT Pi = 3.14159

INPUT Temperature

OUTPUT "Hello"

IF Temperature > 30 THEN

OUTPUT "Hot"

ELSE

OUTPUT "Cold"

ENDIF

- ☐ Stepwise refinement: Break complex problem into smaller sub-problems.

Example: Calculate class average

- Level 1: Get marks → Calculate average → Display result
- Level 2: Initialise total → For each student, input mark, add to total → Divide by count
- ☐ IPO Chart: Input | Process | Output table for planning.

Activity (10 min): Write pseudocode for: "Find largest of 3 numbers." Peer review for clarity.

Plenary (5 min): "Why use pseudocode instead of Python?" Discuss: language-independent, exam requirement, focus on logic.

Homework: IPO chart + pseudocode for: "Calculate total price including 8% tax."

Bilingual Keywords:

- ☐ Algorithm 算法 (suànfǎ)
- ☐ Pseudocode 伪代码 (wěi dàimǎ)
- ☐ Stepwise refinement 逐步求精 (zhúbù qiújīng)
- ☐ IPO chart 输入-处理-输出表 (shūrù-chǔlǐ-shūchū biǎo)

Lesson 2 (40 min): Flowcharts & Structure Diagrams 流程图与结构图

Learning Objectives:

- ☐ Draw flowcharts using standard symbols 使用标准符号绘制流程图
- ☐ Convert between pseudocode and flowcharts 在伪代码和流程图之间转换
- ☐ Use structure diagrams for modular design 使用结构图进行模块化设计

Starter (5 min): Show flowchart symbols. Match: oval, rectangle, diamond, parallelogram. What does each mean?

Main Activity (20 min):

- ☐ Flowchart symbols (CIE standard):
 - Oval: Start/End 椭圆: 开始/结束
 - Rectangle: Process 矩形: 处理
 - Diamond: Decision 菱形: 判断
 - Parallelogram: Input/Output 平行四边形: 输入/输出
 - Rectangle with bars: Predefined process (subroutine) 双边矩形: 预定义过程
- ☐ Flowchart for: "Input number, check if positive or negative, output result."
- ☐ Structure diagrams: Tree showing hierarchy of modules. Top = main program, branches = subroutines.

Example: Payroll system → Input data → Calculate pay → Calculate deductions → Output payslip

Activity (10 min): Convert pseudocode (from Lesson 1) to flowchart. Swap with partner for verification.

Plenary (5 min): "When is a flowchart better than pseudocode? When worse?" Discuss: visual clarity vs space, complex logic.

Homework: Draw structure diagram for "School Report Generator." Include at least 4 sub-modules.

Differentiation:

Bronze: Complete partially drawn flowchart

Silver: Draw flowchart from written description

Gold: Draw flowchart with nested decisions and loops; justify symbol choices

Lesson 3 (40 min): Programming Constructs Review 编程结构复习

Learning Objectives:

- ☐ Review variables, operators, selection, iteration 复习变量、运算符、选择、循环
- ☐ Write simple programs in pseudocode 用伪代码编写简单程序
- ☐ Identify constructs in given code 识别给定代码中的结构

Starter (5 min): Quick recall: "What are the 3 basic programming constructs?" Sequence, Selection, Iteration.

Main Activity (20 min):

- ☐ Variables: Named storage locations. DECLARE Name : STRING. 变量: 命名的存储位置。
- ☐ Data types: INTEGER, REAL, STRING, CHAR, BOOLEAN, DATE.
- ☐ Operators: +, -, *, /, DIV, MOD, &, AND, OR, NOT, =, <>, <, >, <=, >=
- ☐ Selection: IF...THEN...ELSE...ENDIF, CASE...OF...ENDCASE
- ☐ Iteration: FOR...TO...STEP...ENDFOR, WHILE...DO...ENDWHILE, REPEAT...UNTIL
- ☐ Practical: Write pseudocode for:
 - Countdown from 10 to 1 (FOR loop)
 - Keep asking password until correct (REPEAT...UNTIL)
 - Menu system with 4 options (CASE...OF)

Activity (10 min): Code tracing: Given pseudocode, trace variable values. Identify final output.

Plenary (5 min): "What's the difference between WHILE and REPEAT...UNTIL?" Discuss: pre-test vs post-test, minimum iterations.

Homework: Complete 5 pseudocode tracing exercises. Check answers with partner.

Common Misconceptions:

- ☐ ☐ "= means equals in programming" — In pseudocode, = is assignment, not comparison (use == or describe context)
- ☐ ☐ "FOR loops can only count up" — Can count down with negative STEP
- ☐ ☐ "REPEAT...UNTIL continues while condition true" — It continues UNTIL condition is true (stops when true)

Week 18: End-of-Semester Assessment & Review 学期末评估与复习

End-of-Semester Mock Examination (Week 18)

Format: Full Paper 1 (1 hour 30 minutes, 75 marks)

Section A: Short Answer (25 marks)

- ☐ Q1: Number systems and conversions (5 marks)
- ☐ Q2: File size calculations (image, sound) (5 marks)
- ☐ Q3: Compression methods (4 marks)
- ☐ Q4: Network hardware identification (4 marks)
- ☐ Q5: TCP/IP layer matching (4 marks)
- ☐ Q6: Assembly instruction identification (3 marks)

Section B: Structured Questions (50 marks)

- ☐ Q7: Packet switching explanation with diagram (6 marks)
- ☐ Q8: FDE cycle trace table (8 marks)
- ☐ Q9: Memory management — paging vs segmentation (8 marks)
- ☐ Q10: Compiler stages — lexical and syntax analysis (8 marks)
- ☐ Q11: Security scenario — recommend measures (10 marks)
- ☐ Q12: Ethics essay — AI and employment (10 marks)

Conditions: Formal exam conditions, GIE-style paper, externally sourced questions where possible.

Post-Assessment:

- ☐ W18 L2: Review with mark scheme. Students identify own errors.
- ☐ W18 L3: Intervention planning for Semester 2. Set targets.
- ☐ 考后设置下学期目标, 制定干预计划。

Semester 1 Completion Checklist

For Students:

- ☐ I can convert between binary, denary, and hex
- ☐ I can perform two's complement calculations
- ☐ I can calculate file sizes for images and sound
- ☐ I can explain lossy and lossless compression with examples
- ☐ I can describe client-server and P2P networks
- ☐ I can explain packet switching and the TCP/IP stack
- ☐ I can trace the FDE cycle with registers
- ☐ I can explain memory hierarchy and cache
- ☐ I can write and trace simple assembly programs
- ☐ I can explain OS functions and memory management
- ☐ I can compare compilers and interpreters
- ☐ I can identify malware types and prevention measures
- ☐ I can explain encryption, digital signatures, and certificates
- ☐ I can discuss software licensing and AI ethics
- ☐ I can write algorithms in pseudocode and flowcharts

For Teachers:

- ☐ All assessments marked and recorded
- ☐ Intervention groups identified
- ☐ Parent reports written
- ☐ Semester 2 preparation completed

Victoria Park Academy Shenzhen — A-Level Computer Science Curriculum 2024 – 2026